TransitOps Backend Schema

Backend Schema Document

TransitOps — Database & Data Validation Schema Reference

Version: 1.0 **Database:** MongoDB 6.0 (via Mongoose 8.4 ODM) **Companion to:** TransitOps PRD, TRD, ER Diagram

---

1. Schema Conventions (applied platform-wide)

1.1 Base Schema Plugin (`utils/applyBaseSchema.js`)

Applied to **User, Vehicle, Driver, Trip, FuelLog, Expense, Maintenance** (not applied to `Notification` or `AuditLog`). It injects:

Feature	Mechanism	Effect
Audit fields	`schema.add({ createdBy, updatedBy })`	Both `ObjectId` refs to `User` , populated by controllers/services on write
Soft delete	`mongoose-delete` plugin (`deletedAt: true, deletedBy: true, overrideMethods: 'all'`)	Adds `deleted` (Boolean), `deletedAt` (Date), `deletedBy` (ObjectId). Deleting a document sets these fields instead of removing the row; all standard find methods are overridden to exclude soft-deleted documents by default
Optimistic concurrency	`mongoose-update-if-current` plugin	Guards against lost updates on concurrent writes using the document's version key
Query performance	`schema.index({ createdAt: -1 })` , `schema.index({ updatedAt: -1 })`	Added to every base-schema model for recency-sorted queries

1.2 Global Mongoose Options

All schemas declare `{ timestamps: true }` , so every document additionally carries Mongoose-managed `createdAt` / `updatedAt` Date fields.

1.3 Two-Layer Validation

1. **Request layer** — Zod schemas (`validators/entityValidators.js` , `validators/authValidator.js`) validate incoming JSON before it reaches the controller. On `PUT` / `PATCH` , the schema is used in `.partial()` mode (all fields optional) to support partial updates.
2. **Persistence layer** — Mongoose schema-level `required` , `enum` , `min` , `unique` , `trim` validators run on `save()` / `create()` as the final safeguard.

---

2. Collection: `users`

File: `models/User.js` · **Base schema:** ✅ applied

Field	Type	Constraints
`name`	String	required, trim
`email`	String	required, **unique**, indexed, regex-validated, trim

`phone`	String	required, trim
`passwordHash`	String	required — bcrypt hash (never the raw password)
`role`	String (enum)	required — `fleet_manager`, `dispatcher`, `safety_officer`, `financial_analyst`, `viewer`
`createdBy`, `updatedBy`	ObjectId → User	from base schema
`deleted`, `deletedAt`, `deletedBy`	soft-delete fields	from base schema
`createdAt`, `updatedAt`	Date	timestamps

Hooks / Methods - `pre('save')` — if `passwordHash` was modified, re-hashes it with `bcrypt.genSalt(12)` before persisting. - `comparePassword(enteredPassword)` — instance method, `bcrypt.compare()` against the stored hash.

Request-layer validation (Zod, register) `name` ≥ 2 chars · `email` valid email · `password` ≥ 8 chars (hashed into `passwordHash` before save) · `phone` required · `role` optional enum (subset: excludes `viewer` at registration).

---

3. Collection: `vehicles`

File: `models/Vehicle.js` · **Base schema:** ✅ applied · **Extra indexes:** `status`, `vehicleType`

Field	Type	Constraints
`name`	String	required, trim
`registrationNumber`	String	required, **unique**, uppercase, trim
`vehicleType`	String (enum)	required — `truck`, `van`, `bus`, `pickup`, `trailer`, `tanker`
`fuelType`	String (enum)	default `diesel` — `diesel`, `gasoline`, `cng`, `electric`, `hybrid`
`maxLoadCapacity`	Number	required, min 0 (kg)
`odometer`	Number	default 0, min 0
`acquisitionCost`	Number	default 0
`maintenanceInterval`	Number	default 10000
`nextMaintenanceMileage`	Number	default 10000
`region`	String	default `'Central'`, trim
`status`	String (enum)	default `available` — `available`, `on_trip`, `in_shop`, `retired`
`notes`	String	trim

Request-layer validation (Zod) `registrationNumber`, `name` required · `maxLoadCapacity` ≥ 1 · `acquisitionCost`, `odometer` ≥ 0 · `vehicleType`, `fuelType`, `status` optional enums.

Lifecycle note: `status` is mutated by the system (not just the user) — `TripService` sets it to `on_trip` / `available`, `MaintenanceController` gates transitions to/from `in_shop`.

---

4. Collection: `drivers`

File: `models/Driver.js` · **Base schema:** ✅ applied · **Field encryption:** ✅ (`mongoose-field-encryption`)

Field	Type	Constraints
`name`	String	required, trim
`licenseNumber`	String	required, **unique**, uppercase, trim, **encrypted at rest (AES-256-CBC)**

`licenseCategory`	String (enum)	required — `A`, `B`, `C`, `D`, `E`
`licenseExpiryDate`	Date	required
`contactNumber`	String	required, trim, **encrypted at rest (AES-256-CBC)**
`safetyScore`	Number	0–100, default 100
`status`	String (enum)	default `available` — `available`, `on_trip`, `off_duty`, `suspended`
`paymentType`	String (enum)	default `monthly` — `monthly`, `per_trip`
`paymentAmount`	Number	required, min 0, default 0
`kyc.aadharNumber`	String	required, trim (nested object)
`kyc.aadharVerified`	Boolean	default false
`notes`	String	trim

Encryption detail: `DriverSchema.plugin(mongooseFieldEncryption, { fields: ['licenseNumber', 'contactNumber'], secret: process.env.ENCRYPTION_SECRET || 'secret-key-123' })`. Each encrypted field gets two shadow fields (`__enc_<field>`, `__enc_<field>_d`) marking whether it's encrypted and holding the ciphertext for non-string payloads; plaintext is auto-decrypted on document load (`post('init')`) and re-encrypted on save.

Request-layer validation (Zod) `name`, `licenseNumber` required · `paymentAmount` ≥ 0 · `safetyScore` 0–100 · `status` enum here allows `on_leave` (Zod) vs. `off_duty` (Mongoose schema) — **discrepancy noted in §7**.

---

5. Collection: `trips`

File: `models/Trip.js` · **Base schema:** ✅ applied · **Extra indexes:** `status`, `scheduledDate` (desc)

Field	Type	Constraints
`tripCode`	String	**unique**, auto-generated (`TRIP-<8 hex chars>` from `crypto.randomUUID()`) on first save if absent
`source`	String	required, trim
`destination`	String	required, trim
`vehicle`	ObjectId → Vehicle	required
`driver`	ObjectId → Driver	required
`cargoWeight`	Number	required, min 0 (kg)
`plannedDistance`	Number	default 0
`actualDistance`	Number	default 0
`fuelConsumed`	Number	default 0
`revenue`	Number	default 0
`scheduledDate`	Date	default now
`completedAt`	Date	set on completion
`eta`	Date	used for scheduling-conflict checks
`priority`	String (enum)	default `medium` — `low`, `medium`, `high`, `urgent`
`status`	String (enum)	default `draft` — `draft`, `assigned`, `loading`, `dispatched`, `in_transit`, `delivered`, `completed`, `delayed`, `cancelled`
`notes`	String	trim

Request-layer validation (Zod) `source`, `destination` required · `vehicle`, `driver` must match a 24-char hex ObjectId pattern · `cargoWeight` ≥ 0 · optional fields validated when present.

Business-rule validation (service layer, not schema layer) — see Backend Schema §7 cross-reference and the TRD §5.1 for the full rule set (BR-1, BR-2, BR-4, BR-5, scheduling-conflict, terminal-state lock).

---

6. Supporting Collections

6.1 `fuellogs` (`models/FuelLog.js`) — Base schema: ✓

Field	Type	Constraints
`vehicle`	ObjectId → Vehicle	required
`trip`	ObjectId → Trip	optional, default null
`date`	Date	default now, required
`liters`	Number	required, min 0
`costPerLiter`	Number	default 0
`cost`	Number	required, min 0
`station`	String	trim

6.2 `expenses` (`models/Expense.js`) — Base schema: ✓

Field	Type	Constraints
`vehicle`	ObjectId → Vehicle	required
`trip`	ObjectId → Trip	optional, default null
`expenseType`	String (enum)	required — `toll`, `insurance`, `tax`, `fine`, `parking`, `other`
`amount`	Number	required, min 0
`date`	Date	default now, required
`description`	String	trim

6.3 `maintenances` (`models/Maintenance.js`) — Base schema: ✓

Field	Type	Constraints
`vehicle`	ObjectId → Vehicle	required
`description`	String	required, trim
`maintenanceType`	String (enum)	default `corrective` — `preventive`, `corrective`, `inspection`, `emergency`
`cost`	Number	default 0, min 0
`dateStart`	Date	default now
`dateEnd`	Date	set on close
`status`	String (enum)	default `open` — `open`, `closed`
`notes`	String	trim

6.4 `notifications` (`models/Notification.js`) — Base schema: ✗ not applied

Field	Type	Constraints
`userId`	ObjectId → User	optional (supports system-wide broadcasts)

`title`	String	required
`message`	String	required
`type`	String (enum)	default `info` — `info`, `warning`, `error`, `success`
`read`	Boolean	default false
`referenceId`	ObjectId	polymorphic — may reference a `Vehicle`, `Trip`, etc.; no `ref` enforced at schema level

(Only `{ timestamps: true }` is applied — no soft delete/audit fields, since notifications are transient.)

6.5 `auditlogs` (`models/AuditLog.js`) — Base schema: ✕ not applied

Field	Type	Constraints
`action`	String (enum)	required — `CREATE`, `UPDATE`, `DELETE`, `RESTORE`
`modelName`	String	required — name of the model being audited
`documentId`	ObjectId	required — polymorphic reference to the affected document
`userId`	ObjectId → User	actor who performed the action
`changes`	Mixed	required — freeform payload of what changed

(Deliberately excluded from soft-delete/base schema — audit records must be immutable and permanent.)

---

7. Cross-Cutting Notes & Discrepancies

#	Observation
1	**Driver status enum mismatch:** Mongoose schema (`models/Driver.js`) allows `off_duty`; the Zod request validator (`validators/entityValidators.js`) allows `on_leave` for the same field. One of the two should be corrected to match — as written, a request with `status: "on_leave"` would pass request validation but fail Mongoose persistence validation (and vice versa for `off_duty`).
2	**Role enum drift:** `constants/index.js` defines `ROLES` differently (`admin`, `dispatcher`, `fleet_manager`, `driver`, `analyst`) from the `User` schema's actual `role` enum (`fleet_manager`, `dispatcher`, `safety_officer`, `financial_analyst`, `viewer`). The schema enum is authoritative in production; the constants file should be reconciled (also flagged in the TRD).
3	**Polymorphic references aren't schema-enforced:** `Notification.referenceId` and `AuditLog.documentId` are plain `ObjectId` fields with no `ref`, so Mongoose cannot `.populate()` them automatically — the consuming code must know which collection to look up based on context (e.g. `AuditLog.modelName`).
4	**Soft delete is opt-in per collection:** `User`, `Vehicle`, `Driver`, `Trip`, `FuelLog`, `Expense`, `Maintenance` all soft-delete (row stays, `deleted: true`). `Notification` and `AuditLog` do not have delete endpoints exposed at all in the current routes, so this is consistent with their transient/permanent-record nature respectively.
5	**Optimistic concurrency (`mongoose-update-if-current`)** is active on all base-schema collections — concurrent

	conflicting updates to the same document will be rejected rather than silently overwritten, which the frontend/service layer must handle (retry or surface a conflict error) though no explicit conflict-handling code was found in `tripService.js` at the time of writing.
6	**Encryption key fallback risk:** `Driver` model falls back to a hardcoded `'secret-key-123'` if `ENCRYPTION_SECRET` is unset — flagged as a required production hardening step in the TRD (§8.1).

8. Index Summary (all collections)

Collection	Indexes
`users`	`email` (unique), `createdAt` desc, `updatedAt` desc
`vehicles`	`registrationNumber` (unique), `status`, `vehicleType`, `createdAt` desc, `updatedAt` desc
`drivers`	`licenseNumber` (unique, on encrypted value), `createdAt` desc, `updatedAt` desc
`trips`	`tripCode` (unique), `status`, `scheduledDate` desc, `createdAt` desc, `updatedAt` desc
`fuellogs`	`createdAt` desc, `updatedAt` desc
`expenses`	`createdAt` desc, `updatedAt` desc
`maintenances`	`createdAt` desc, `updatedAt` desc
`notifications`	*(none beyond default `_id`)*
`auditlogs`	*(none beyond default `_id`)*

Companion documents: PRD, TRD, ER Diagram, UML Class Diagram, Trip Lifecycle Workflow, UI/UX Theme Guide.